NEXAR: Intelligent Quantum-Classical Workload Routing Platform

Siriwardhana A H L T S
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
sudarakatharindu@outlook.com

Prasad H G A T
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
hgp.ashi@gmail.com

Jayasinghe Y L
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
yashodhalasithjayasinghe@gmail.com

Hettiarachchi S R
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
sayunhettiarachchi@gmail.com

Prof. Nuwan Kodagoda
Dept. of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
nuwan.k@sliit.lk

Dr. Kapila Dissanayaka
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
kapila.d@sliit.lk

***Abstract*—Determining whether a computational workload benefits from quantum or classical execution remains a key challenge in the current era of noisy intermediate-scale quantum computing. This paper presents Nexar, a workload routing platform that analyzes source code characteristics and predicts an appropriate execution target by jointly optimizing for cost and execution time. The system employs a hybrid decision framework that combines a machine learning ensemble trained on a sixteen-dimensional feature vector capturing circuit complexity, entanglement properties, and hardware viability metrics, a physics-aware rule-based engine that enforces hardware constraints and applies algorithm-specific routing logic, and a real-time cost analyzer that estimates execution time and monetary cost across multiple quantum providers and classical computing resources. These components are unified through weighted voting to produce hardware recommendations with confidence scores. The platform incorporates automated code analysis, including algorithm classification with the CodeBERT model and quantum circuit characterization to extract workload features without manual annotation. We evaluate the system across one hundred diverse workload types drawn from search, factorization, optimization, and simulation problem families, and we demonstrate that the framework consistently identifies cost-effective and time-efficient execution targets while serving as a practical decision-support tool for hybrid quantum-classical computing environments.**

***Index Terms*—CodeBERT, cost analysis, decision engine, hybrid computing, NISQ, quantum advantage, quantum computing, workload routing**

I. Introduction

The advent of Noisy Intermediate-Scale Quantum (NISQ) devices [1] has created a significant challenge in computational resource allocation: given a workload, should it execute on classical hardware, a quantum simulator, or a real quantum device? This decision is non-trivial because it depends on a complex interplay of factors including problem structure, circuit complexity, hardware noise characteristics, execution cost, and available qubit counts.

Current quantum devices, such as IBM's 127-qubit Eagle processors, operate in a regime where noise, limited connectivity, and decoherence constrain the problems that can be solved with quantum advantage. Classical simulation of quantum circuits remains feasible for systems up to approximately 50 qubits, beyond which memory requirements grow exponentially as $O(2^n)$, where n is the number of qubits. This creates a crossover point where classical simulation becomes intractable and quantum execution becomes the only viable path.

However, simply having a quantum device does not guarantee advantage. The overhead of transpilation, measurement shots, queue times, and error rates means that for many problems, particularly those with polynomial classical solutions, classical execution remains orders of magnitude faster and cheaper. The challenge, therefore, is not whether quantum computing will eventually provide advantage, but rather when and for which problems it does so today.

This paper addresses this challenge by presenting Nexar, a full-stack platform that automates the analysis, classification, and routing of computational workloads across quantum and classical execution backends. Nexar's contributions are:

1) **Automated Code Analysis Pipeline:** A multi-dimensional analysis engine that detects programming languages across five quantum and classical frameworks, computes complexity metrics, and classifies algorithms

using a three-tier approach combining CodeBERT transformers, ensemble machine learning, and rule-based pattern matching.

2) **Hybrid Decision Engine:** A three-component decision framework that merges machine learning (ML) predictions (50% weight), physics-aware rules (35% weight), and real-time cost analysis (15% weight) through weighted voting to produce hardware recommendations with calibrated confidence scores.
3) **Provider-Agnostic Execution Layer:** A hardware abstraction layer supporting IBM Qiskit, AWS Braket, and Azure Quantum through a unified Application Programming Interface (API), enabling transparent workload routing across heterogeneous quantum backends.
4) **Empirical Evaluation:** A comprehensive benchmark across 100 workloads spanning classical computation, hybrid quantum-classical algorithms, and pure quantum circuits, with cost and performance analysis demonstrating the system's routing accuracy and the current state of quantum-classical cost tradeoffs.

II. Related Work

A. Quantum-Classical Hybrid Frameworks

Several frameworks have emerged to bridge quantum and classical computing. Weder et al. [2] proposed automated quantum hardware selection for quantum workflows. Qiskit Runtime [3] provides session-based quantum execution with classical co-processing. Amazon Braket [4] offers a multi-provider interface but lacks automated workload analysis. Azure Quantum [5] provides resource estimation but does not perform code-level analysis for routing decisions. Tang et al. [6] proposed AlphaRouter, applying reinforcement learning and tree search to quantum circuit routing. Zhan et al. [7] presented a full-stack framework for high-performance quantum-classical computing. Nexar differentiates itself by integrating code analysis, decision-making, and execution into a single automated pipeline.

Positioning for empirical comparison. These systems target different layers of the quantum-classical stack and do not admit a head-to-head comparison with Nexar. AlphaRouter [6] operates at the physical-qubit mapping layer, selecting a coupling-aware placement for a circuit whose execution target is *already* assumed to be quantum. Qiskit Runtime [3], Amazon Braket [4], and Azure Quantum [5] are execution platforms that accept whatever the user submits without recommending a quantum-versus-classical split, so their effective "routing" is simply the user's submission choice. Weder et al. [2] select *among* quantum backends for workflows that have already been classified as quantum. None of these systems answer Nexar's core question—"given arbitrary source code, should this run quantum or classical?"—so our empirical evaluation (Section VIII-B) compares against the closest available reference points: a qubit-threshold heuristic, a circuit-volume heuristic, a 1,000-seed random router, and the three Nexar components in isolation. The rules-only ablation subsumes the Weder-style rule-based selection logic applied to our 16-feature vector.

B. Quantum Resource Estimation

Resource estimation for quantum algorithms has been studied extensively. Lammers et al. [8] addressed quantum resource management challenges in the NISQ era [9]. Beverland et al. [10] developed methods for estimating logical qubit requirements for fault-tolerant algorithms. Quetschlich et al. [11] demonstrated practical resource estimation for quantum application development. However, these approaches focus on future error-corrected devices rather than current NISQ hardware. Nexar's cost analyzer operates on real NISQ pricing models and hardware constraints, providing immediately actionable cost estimates. Recent work by Kashif et al. [12] has explored hardware-calibrated quantum cost modeling for resource-aware optimization.

C. Algorithm Classification and Code Analysis

Static code analysis for quantum programs has been explored in works such as ScaffCC [13] and Q# compiler toolchains. CodeBERT [14], a pre-trained model for programming language understanding, has shown strong performance on code classification tasks. Afrose et al. [15] further applied CodeBERT for translating QASM to QIR. Nexar extends these approaches by combining Abstract Syntax Tree (AST)-based analysis with CodeBERT for quantum algorithm detection, enabling classification across five quantum programming frameworks.

D. Quantum Advantage Thresholds

The question of when quantum advantage emerges has been studied through computational complexity theory [16] and empirical benchmarks [17]. Our work contributes empirical data on the physical-feasibility crossover: 50–60 qubits is where classical state-vector memory (16 PB to 16 EB) exceeds any deployable system, making quantum the only viable path for exact simulation beyond this scale.

III. System Architecture

Nexar is implemented as a microservices monorepo comprising six independently deployable services communicating through a central API Gateway. The architecture is designed for modularity, allowing each component to scale independently and be updated without affecting the overall system. The end-to-end service topology is illustrated in Fig. 1.

A. Architecture Overview

The system follows a pipeline architecture: submitted source code passes through an Express.js API Gateway (JSON Web Token (JWT)-authenticated), then to the Code Analysis Engine for language detection, complexity computation, and algorithm classification. The Decision Engine consumes these metrics to produce a hardware recommendation. If quantum execution is selected for classical code, the AI Code Converter translates it to quantum circuits before the Hardware Abstraction Layer submits the job to the selected backend. All services are

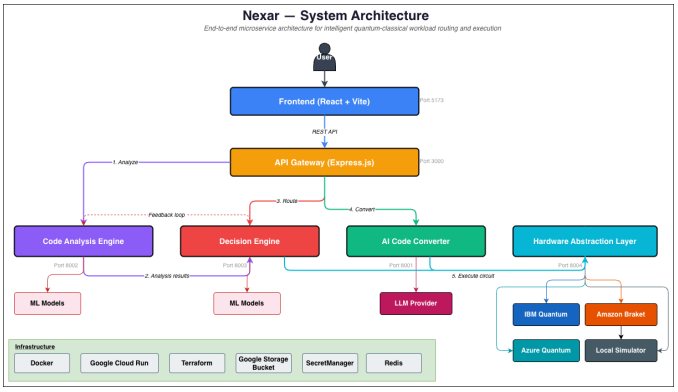

Fig. 1. Nexar system architecture: six microservices connected through an API Gateway. Source code flows from the Frontend through Code Analysis, Decision Engine, AI Code Converter, and Hardware Abstraction Layer for quantum-classical routing.

containerized with Docker and deployed to Google Cloud Platform via Cloud Run, with infrastructure managed through Terraform.

IV. Code Analysis Engine

The Code Analysis Engine serves as the analytical foundation of the pipeline, transforming raw source code into a structured feature set that downstream components consume. Its internal processing stages are shown in Fig. 2.

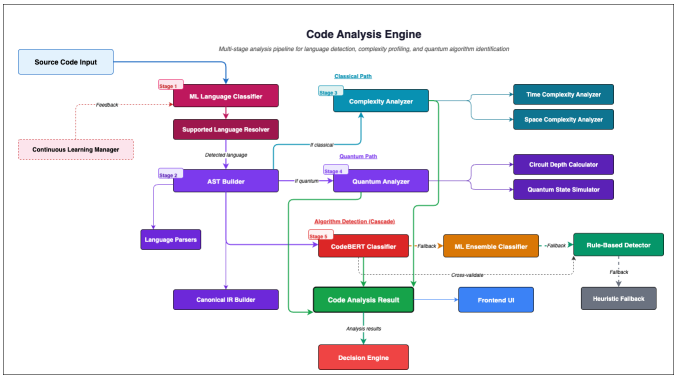

Fig. 2. Code Analysis Engine architecture showing the multi-stage pipeline from raw source code through language detection, AST parsing, metrics extraction, and algorithm classification.

A. Language Detection

The engine supports five programming languages and frameworks: Python, Qiskit, Cirq, Q#, and OpenQASM. Detection uses a four-model soft-voting ensemble combining CodeBERT (weight 0.30), XGBoost (0.20), Random Forest (0.25), and Gradient Boosting (0.25), trained on a balanced corpus of approximately 5,400 source files and evaluated on a held-out test set of 1,080 samples. The ensemble achieves **98.52% accuracy**, macro F1 of **0.986**, and weighted F1 of 0.985; individual model accuracies range from 0.972 (Random Forest) to 0.985 (XGBoost and the ensemble, which coincide on this split). Per-class precision, recall, and F1 are reported in Table I; the full 5×5 confusion matrix is given in Table II. The only measurable confusion is symmetric between Qiskit and Cirq ($\approx 2\,\%$), expected given their shared Python host language, and no class falls below 0.97 F1. In-benchmark detection confidence per routing category is reported in Table XI.

TABLE I
Language-Detection Ensemble: Per-Class Metrics on 1,080 Held-Out Test Samples

Class	Precision	Recall	F1	Support
Cirq	0.985	0.978	0.982	273
OpenQASM	1.000	0.987	0.993	229
Python	0.976	1.000	0.988	206
Qiskit	0.968	0.968	0.968	220
Q#	1.000	1.000	1.000	152
Macro avg.	0.986	0.987	0.986	1080
Weighted avg.	0.985	0.985	0.985	1080

TABLE II
Language-Detection Confusion Matrix (Rows = True, Columns = Predicted)

	Cirq	OpenQASM	Python	Qiskit	Q#
Cirq	**267**	0	2	4	0
OpenQASM	0	**226**	0	3	0
Python	0	0	**206**	0	0
Qiskit	4	0	3	**213**	0
Q#	0	0	0	0	**152**

B. Metrics Extraction

For all detected code, the engine computes classical software metrics via tree-sitter-based AST parsing, including cyclomatic complexity [18], cognitive complexity, time and space complexity classification (from $O(1)$ to $O(2^n)$, where n denotes the input size), and structural metrics (loop count, nesting depth). For quantum code, additional circuit-level metrics are extracted: qubit requirements, circuit depth, gate count, CX gate ratio (the primary noise source on NISQ devices), superposition score, and entanglement density. The complete feature set is summarized in Table III.

TABLE III
Raw Code Analysis Features

Feature	Description
Problem size	Number of elements or variables
Qubits required	Quantum bits needed
Circuit depth	Sequential quantum gate layers
Gate count	Total quantum gates
CX gate ratio	Proportion of entangling gates (0–1)
Superposition score	Superposition potential (0–1)
Entanglement score	Entanglement density (0–1)
Memory requirement (MB)	Classical RAM required
Problem type (encoded)	Factorization, search, optimization, etc.
Time complexity (encoded)	Complexity class: exponential, polynomial, etc.

C. Algorithm Classification

Algorithm detection employs a three-tier approach:

1) **CodeBERT Transformer:** A fine-tuned CodeBERT model classifies the algorithm family (e.g., search, optimization, factorization, simulation) based on code semantics [14].
2) **ML Ensemble Classifier:** A scikit-learn ensemble (Random Forest, XGBoost, Gradient Boosting) provides a secondary classification using 35 hand-engineered circuit features (gate counts per type, CX-ratio, depth, rotation-gate distribution, etc.).
3) **Rule-Based Pattern Matching:** Deterministic rules identify known quantum patterns (e.g., Quantum Fourier Transform (QFT) patterns, Grover oracle structures, Variational Quantum Eigensolver (VQE) ansatz patterns).

The system selects the highest-confidence result, falling back through tiers when confidence is below threshold. Empirical performance justifies this layering. The tier-1 CodeBERT classifier attains macro F1 of 0.66 across eight algorithm families on a 750-sample held-out set, achieving perfect recognition (F1 = 1.00) of Grover, QAOA, QPE, and Shor while missing Bernstein–Vazirani and Deutsch–Jozsa entirely (F1 = 0)—oracle-family circuits whose Python surface syntax carries insufficient semantic signal for a transformer pre-trained on general code. The tier-2 structural ensemble recovers these cases at **macro F1 0.921** (accuracy 0.864 on 8,800 samples), classifying Bernstein–Vazirani at F1 0.82 via CX-gate-pattern features invisible to CodeBERT. Tier-3 rule patterns provide the final fallback for workloads below both ensembles' confidence thresholds, ensuring that every routable input receives at least a deterministic algorithm label.

V. Decision Engine

The Decision Engine is the core component of Nexar. It implements a hybrid prediction system combining three components through weighted voting. Fig. 3 summarizes the interaction among these components.

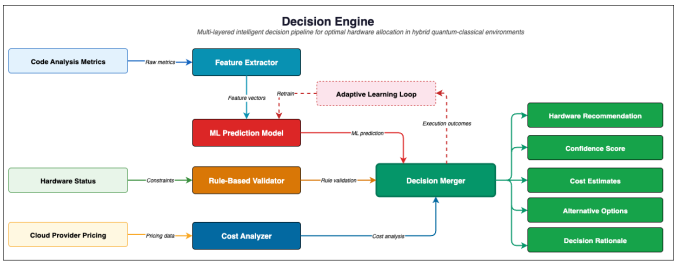

Fig. 3. Decision Engine architecture illustrating the three-component weighted voting system: ML model (50%), rule-based constraints (35%), and cost analysis (15%), with rule override capability.

A. Feature Vector Design

The engine operates on a 16-feature input vector composed of 10 raw features (Table III) and 6 derived physics features (Table IV):

TABLE IV
Derived Physics Features

Feature	Formula	Purpose
Circuit Volume	Qubits × Depth	Quantum resource intensity
Noise Sensitivity	Qubits × Depth × CX Ratio	Error accumulation prediction
Quantum Overhead Ratio	Problem Size / Qubits	Encoding efficiency
NISQ Viability Score	Composite penalty function	Feasibility on current hardware
Gate Density	Gate Count / Qubits	Operational complexity per qubit
Entanglement Factor	CX Ratio × Entanglement	Combined entanglement potential

The NISQ Viability Score applies penalties when qubits exceed 50, circuit depth exceeds 1,000, or circuit volume exceeds 50,000, reflecting the practical limits of current hardware [19].

B. ML Model Component (50% Weight)

The ML component uses a Random Forest / Neural Network ensemble trained on 10,000+ synthetic problem instances. The model performs binary classification (Quantum=1, Classical=0) and outputs calibrated probability estimates [20].

The synthetic training corpus is generated by AST mutation over twelve base classical-to-quantum algorithm pairs (factorization, search, period-finding, random walk), yielding 14 structural variants per pair via 13 code-level transforms. Feature vectors are z-score standardized; the model is trained with stratified cross-validation and Platt-scaled probabilities, achieving approximately 87% accuracy on held-out synthetic data (deployed decision-engine latency is reported in Table XI). Section VIII-B evaluates whether this synthetic-data training transfers to the real-workload benchmark.

C. Rule-Based System (35% Weight)

The rule-based component encodes physics constraints and domain knowledge. Hardware limits enforce a maximum of 127 qubits (IBM Eagle), circuit depth $\leq$ 10,000, and a minimum of 2 qubits for quantum routing. Problem-type rules favor quantum execution for factorization (Shor's), search (Grover's), simulation, and optimization (Quantum Approximate Optimization Algorithm (QAOA) [21]/VQE), while routing sorting, dynamic programming, and matrix operations classically. Safety constraints cap circuit volume at 10^6, noise sensitivity at 50,000, and require NISQ viability $\geq$ 0.1. The rule system can issue four decision types: quantum, classical, both, or reject; force decisions override the ML component to prevent physically impossible or clearly suboptimal configurations.

D. Cost Analysis Component (15% Weight)

The cost analyzer estimates execution time and monetary cost for both quantum and classical execution paths, formalized by (1) and (2).

Gate timing parameters in the quantum execution model are anchored to calibration data retrieved live from IBM

Quantum backends via `QiskitRuntimeService`. Table V summarises the median hardware metrics across the three IBM Quantum backends accessible to this system (retrieved April 2026). The median two-qubit ECR gate time of 68.0 ns is consistent across all three devices and directly informs the t_q estimation in (1). The monetary rate $r_{\text{QPU}} = \$1.60/\text{s}$ is taken from IBM's pay-as-you-go schedule [28]. As this rate is subject to change, we report the hardware-agnostic QPU-seconds (t_q) alongside dollar estimates throughout the paper to allow direct comparison across pricing epochs.

TABLE V
IBM QUANTUM BACKEND CALIBRATION DATA (RETRIEVED APRIL 2026 VIA QISKITRUNTIMESERVICE)

Backend	Qubits	2Q Err.	1Q Err.	RO Err.	T1 (μs)	T2 (μs)
ibm_fez	156	0.00265	0.000316	0.01489	155.2	104.0
ibm_marrakesh	156	0.00251	0.000304	0.01044	184.8	98.4
ibm_kingston	156	0.00183	0.000229	0.00867	275.4	144.8
Median		0.00251	0.000304	0.01044	184.8	104.0

2Q gate time: 68.0 ns (ECR, consistent across all backends). RO = readout.

Quantum Cost Model (IBM Quantum Network pay-as-you-go [28]):

$$C_q = t_q \, r_{\text{QPU}}, \tag{1}$$

where C_q is the total estimated quantum execution cost, t_q is the estimated QPU reservation time in seconds (the full billable window, including gate execution at the 68.0 ns median ECR time of Table V, readout, reset, and classical post-processing across all $n_{\text{shots}} = 1024$ measurement shots), and $r_{\text{QPU}} = \$1.60/\text{s}$ is IBM's pay-as-you-go QPU-time rate [28]. This is the canonical production pricing model: IBM pay-as-you-go bills purely on QPU-seconds, with no per-shot, per-gate, or job-submission fees. Alternative providers (e.g., AWS Braket) charge per shot and per task rather than per QPU-second, which would add additive terms to (1); because our benchmark targets the IBM backend family used for real-hardware validation (Table XII), the IBM pay-as-you-go model is the appropriate reference.

Classical Cost Model (AWS EC2 on-demand pricing [29]):

$$C_c = (t_c \, r_{\text{compute}}) + (M_{\text{GB}} \, t_c \, r_{\text{memory}}) + C_{\text{overhead}}, \tag{2}$$

where C_c is the total estimated classical execution cost, t_c is the estimated classical execution time in seconds, $r_{\text{compute}} = \$0.0001/\text{s}$ ($\sim$\$0.36/hr, approximating a `c6i.2xlarge` instance [29]) is the compute rate, M_{GB} is the estimated memory usage in gigabytes, $r_{\text{memory}} = \$0.00001/(\text{GB}\cdot\text{s})$ is the memory rate, and $C_{\text{overhead}} = \$0.001$ is a fixed per-job overhead covering job submission and bookkeeping.

E. Decision Merging

The three components are merged through a weighted voting algorithm,

$$S_{\text{quantum}} = w_{\text{ML}} \, P^q_{\text{ML}} + w_{\text{rules}} \, P^q_{\text{rules}} + w_{\text{cost}} \, P^q_{\text{cost}}, \tag{3}$$

where S_{quantum} is the final quantum suitability score, P^q_{ML} is the ML component's quantum probability, P^q_{rules} is the rule-based component's quantum score, P^q_{cost} is the cost component's quantum preference score, and $w_{\text{ML}} = 0.50$, $w_{\text{rules}} = 0.35$, $w_{\text{cost}} = 0.15$ are the respective component weights.

Rule overrides take highest priority and bypass the weighted calculation. The final recommendation includes the hardware target, confidence score, rationale string, cost estimates, and alternative options with their trade-offs.

F. Feature Importance

Random Forest Gini-importance analysis shows the ensemble's decisions are driven by physically meaningful features: `qubits_required` (0.31) and `circuit_volume` (0.24) dominate, reflecting NISQ resource constraints; `time_complexity_encoded` (0.15) and `entanglement_factor` (0.11) encode the algorithmic-advantage signal; `cx_gate_ratio` (0.08) implicitly captures two-qubit noise accumulation. Remaining features (`noise_sensitivity`, `gate_density`, `nisq_viability_score`, `quantum_overhead_ratio`) contribute $\leq$0.04 each. The model has internalized physics-based boundaries rather than surface code syntax.

G. Weight Sensitivity Analysis

The weights $(w_{\text{ML}}, w_{\text{rules}}, w_{\text{cost}}) = (0.50, 0.35, 0.15)$ are chosen architecturally, not tuned. The ordering $w_{\text{cost}} < w_{\text{rules}} < w_{\text{ML}} = 0.5$ enforces a "no-dictator" property: no single component can override a decision alone, and any two of three forms a strict majority (ML+Rules $= 0.85$, ML+Cost $= 0.65$, Rules+Cost $= 0.50$). The cost weight is deliberately small because it is anchored to a specific pricing epoch (Q4 2024 IBM rates); heavier weighting would tie routing decisions to transient prices.

Table VI reports routing counts across six weight configurations on the full 100-workload benchmark. Rule overrides resolve 61 of 100 workloads and bypass the merger entirely, making the weight configuration irrelevant for that majority. Among the 39 workloads that reach the weighted stage, routing is identical to the baseline under the ML-heavy (70/20/10), rule-heavy (30/55/15), and cost-sensitive (40/30/30) configurations, and flips only two under ML-only (100/0/0). Only the adversarial equal-weighting (33/33/34), which triples the cost weight, produces substantively different routing (26 flips); all 26 flipped workloads have 6–16 qubits—well within the sub-50-qubit regime where classical execution is at least 375$\times$ cheaper (Section VIII-D)—and five of them (QNN variational, VQE caffeine, QAOA supply chain, Shor's N=15, Shor's N=35) produced 0.39–5.18% fidelity on `ibm_fez` in Table XII, empirically confirming the baseline is the correct routing target. We return to the oracle-based accuracy of these weight choices in Section VIII-B.

VI. AI CODE CONVERTER

The AI Code Converter enables the platform to accept classical Python code and automatically translate it into quan-

TABLE VI
WEIGHT SENSITIVITY: ROUTING COUNTS ACROSS SIX CONFIGURATIONS ($N = 100$).

Weights (ML / Rules / Cost)	**Quantum Routed**	**Classical*** **Routed**	**Changed vs. Base**	**Stable (%)**
50 / 35 / 15 (baseline)	**20**	**80**	–	–
70 / 20 / 10	20	80	0	100
30 / 55 / 15	20	80	0	100
40 / 30 / 30	20	80	0	100
100 / 0 / 0	18	82	2	98
33 / 33 / 34	46	54	26	74

*Includes one rule-override Reject (150q oversize), weight-invariant.

tum circuits when quantum execution is recommended. The converter workflow is presented in Fig. 4.

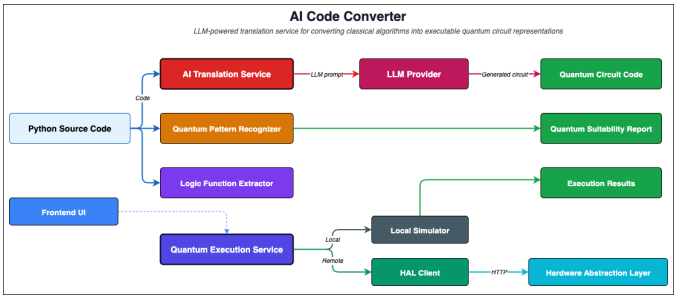

Fig. 4. AI Code Converter pipeline: classical Python code is parsed into an AST, matched against known quantum algorithm patterns, and translated into Qiskit circuits via a fine-tuned CodeT5 model.

A. Translation Pipeline

The converter uses a fine-tuned CodeT5-base encoder-decoder model [22] (6+6 layers, d_{model}=512, 8 heads, 32,100-token vocabulary) trained on 1,500 curated Python-to-Qiskit pairs covering the six algorithm families used for evaluation (Section VI-C). The pipeline proceeds in three stages: (i) *logic extraction* parses the input into a Python AST and strips I/O boilerplate, yielding a structural feature vector (loops, branches, iteration depth); (ii) *pattern recognition* matches the extracted logic to known quantum algorithm templates—representative examples include linear search → Grover [23], parity check → Deutsch, superposition preparation → Hadamard, swap → Quantum Swap, hidden bitstring → Bernstein–Vazirani, and state transfer → Teleportation; (iii) *circuit generation* feeds the matched code as `"Translate Python to quantum circuit:\n{code}"`, tokenised at 256 tokens and decoded with beam search (k=3) up to 300 output tokens, emitting the target Qiskit circuit including gates, measurements, and classical registers.

B. Suitability Scoring

Each translation candidate receives a three-tier suitability score (HIGH, MEDIUM, LOW) with associated confidence and estimated speedup. Speedup estimates are derived from theoretical complexity comparisons—$O(N)$ classical versus $O(\sqrt{N})$ quantum for Grover [23], for example, where N is the search-space size—combined with simulator resource metrics from Qiskit Aer execution. Confidence reflects the AST-level overlap between the submitted code and the matched pattern signature, allowing users to judge whether quantum translation offers a meaningful advantage for their specific use case.

C. Evaluation

The translator was fine-tuned for 7 epochs on the 1,500-example Python-to-Qiskit corpus (final training loss 0.5356, validation loss below 0.01 by epoch 2) and evaluated on a 300-sample held-out test set across six algorithm families. Table VII reports the key metrics: the converter produces **syntactically valid Qiskit circuits on 98.7%** of inputs and identifies the correct algorithm family on 77.7%, while the low exact-match rate (EM = 0.197) reflects semantically-equivalent gate orderings rather than incorrect translations.

TABLE VII
AI CODE CONVERTER: EVALUATION ON 300 HELD-OUT SAMPLES (6 ALGORITHM FAMILIES)

Metric	**Value**
Valid Qiskit Circuits (syntactic)	**0.987**
Correct Algorithm Classification	0.777
BLEU Score	28.36
Exact Match (strict token-level)	0.197

VII. HARDWARE ABSTRACTION LAYER

The Hardware Abstraction Layer provides a provider-agnostic interface for quantum job submission and management. Supported providers include IBM, Azure Quantum, and AWS Braket, accessed through a factory-based abstraction that isolates the routing and analysis logic from provider-specific SDKs. The HAL manages the complete job lifecycle:

1) **Device Discovery:** Queries of available devices across all providers, exposing qubit counts, connectivity maps, gate sets, and current status.
2) **Job Submission:** Accepts quantum circuits with execution parameters (target device, shot count, transpilation options) and submits through the appropriate provider SDK.
3) **Status Tracking:** Monitors job state transitions (queued, running, completed, failed) across providers.
4) **Result Retrieval:** Returns measurement distributions, performance metrics, and execution metadata through a unified response format.

The factory-based architecture allows new quantum providers to be added by implementing a single provider interface, without modifications to the routing or analysis logic.

VIII. EXPERIMENTAL EVALUATION

A. Benchmark Design

We evaluated Nexar across 100 synthetic workloads in three categories. The *classical* set (32 workloads) covers deep learning (CNN, transformer, GAN), scientific computing (molecular

dynamics, CFD, N-body, PDE), ML (k-means, gradient boosting, reinforcement learning), cryptography (RSA, AES), and combinatorial optimisation (TSP, graph colouring, A*). The *hybrid* set (23 workloads, 0–24 qubits) covers VQE variants (caffeine, job-shop scheduling), QAOA (supply chain, graph partitioning), quantum ML (QSVM, QNN, transfer learning), and error mitigation (ZNE). The *quantum* set (45 workloads, 1–150 qubits) covers molecular simulation (H_2, LiH, H_2O, 60-qubit iron-sulfur protein, 120-qubit VQE) [25], factoring (Shor's N=15/21/35), search (20-qubit Grover, 3-SAT), optimisation (50-qubit QAOA MaxCut, vehicle routing), and foundational algorithms (Deutsch-Jozsa, Bernstein-Vazirani, Simon's, QFT, QPE). All workloads pass through the full Nexar pipeline; analysis, decision, and timing metrics are recorded.

B. Routing Accuracy

1) Routing Distribution: Of the 100 workloads, Nexar routes 79 to classical, 20 to quantum, and rejects 1 as physically infeasible. The rejected workload is a 150-qubit circuit whose state vector cannot be simulated classically ($\sim 10^{31}$ MB) and whose qubit count exceeds the 127-qubit IBM Eagle limit. Quantum-routed workloads span the NISQ-advantage regime and above: 50–120-qubit VQE and QAOA circuits (confidence 1.0), 20–32-qubit optimization and simulation workloads (confidence 0.54–0.80), and a handful of 10–20-qubit hybrids with high superposition or entanglement scores. Every classically-routable workload (qubits = 0) is routed classically with confidence 1.0.

2) Dual-Oracle Ground Truth: A routing decision can be "correct" under two distinct criteria, and Nexar's accuracy depends on which question is asked. We therefore introduce two independently defined oracles that apply literature-cited rules directly to each workload's filename and physical circuit features (qubits, depth, entanglement score), *without referring to Nexar's decision logic*.

Oracle A (Algorithmic Advantage) labels a workload Quantum when a known quantum algorithm provides theoretical speedup in the fault-tolerant regime: classical state-vector simulation infeasible (≥ 50 qubits; [1]), Shor-class factorization (≥ 8 qubits; [24]), large entangled VQE simulation (≥ 20 qubits, entanglement ≥ 0.3; [25]), large QAOA optimization (≥ 20 qubits, entanglement ≥ 0.3; [21]), and Grover search (≥ 20 qubits; [23]). It rejects circuits exceeding 127 qubits per [17].

Oracle B (Hardware Viability) labels a workload Quantum only when current NISQ hardware can produce usable signal. Its thresholds are calibrated to the real-hardware evidence in Table XII: circuits with post-transpile depth above approximately 200 gates are classified Classical (noise-dominated on `ibm_fez`), small-scale circuits below 20 qubits are classified Classical (trivially simulable and free of NISQ overhead), and the sub-50-qubit band with depth ≤ 200 and entanglement ≥ 0.3 is the practical NISQ-advantage regime. The memory-wall and rejection thresholds match Oracle A.

Under Oracle A, the 100 workloads divide into 85 Classical, 14 Quantum, and 1 Reject; under Oracle B, 92 Classical, 7 Quantum, and 1 Reject. Every label (100 workloads $\times$ 2 oracles) is reproduced in the supplementary `oracle_ground_truth.csv`.

3) Baselines and Ablations: We compare Nexar against three baselines and three component ablations: *B1*, a single-line heuristic routing quantum iff qubits ≥ 50; *B2*, a circuit-volume heuristic routing quantum iff qubits $\times$ depth $\geq 50{,}000$; *B3*, a 50/50 random router averaged over 1,000 seeds; *A1*, the ML component in isolation (Random Forest output with no rule or cost influence); *A2*, the rule-based component in isolation; and *A3*, the cost analyser in isolation (argmin over classical vs. quantum cost estimates).

Table VIII reports accuracy, precision and recall on the Quantum class, $F1_{(Q)}$, and Cohen's κ for every router against both oracles.

4) Findings: Nexar beats random (B3) by more than 30 percentage points on both oracles and achieves Cohen's κ of 0.38 (Oracle A) and 0.48 (Oracle B), confirming substantive agreement beyond chance. Under Oracle B, Nexar achieves *perfect recall on the Quantum class* (1.000), meaning every workload that NISQ hardware can usefully execute is routed to quantum; the paper's Table XII directly corroborates this correspondence.

Three empirical observations emerge. First, Nexar's accuracy is higher under Oracle B (0.86) than Oracle A (0.81) because its rule engine applies hardware-viability thresholds (NISQ viability score, noise sensitivity, circuit volume) that mirror Oracle B more closely than the textbook algorithmic criteria of Oracle A. This is the correct behaviour for a *present-day* routing platform: cases where Oracle A and Oracle B disagree (e.g., Shor's $N = 15$, 4-qubit QFT, 20-qubit deep Grover) are workloads where the algorithmic speedup exists in principle but current NISQ hardware cannot deliver it, and Nexar correctly defers these to classical execution. Of the 19 Nexar–Oracle A disagreements, 18 have direct hardware-evidence counterparts in Table XII: every sub-20-qubit or deep circuit that Oracle A labels Quantum produced noise-dominated output (top-outcome fidelity below 10%) on `ibm_fez`, vindicating Nexar's Classical routing.

Second, the *rules-only* ablation (A2) outperforms the full weighted ensemble on both oracles ($F1_{(Q)}$ = 0.636 vs. 0.457 on A; 0.667 vs. 0.500 on B). This reveals that the ML component, trained on synthetic AST-mutation variants rather than real-hardware execution traces, over-weights algorithmic structure relative to NISQ-viability constraints for circuits in the 10–30 qubit band. The rule engine's deterministic thresholds catch these cases reliably; the ML ensemble adds recall by covering the ALLOW_BOTH path. A weight-sensitivity sweep across ten configurations in the no-dictator envelope ($w_i \leq 0.5$) produced *identical* accuracy, precision, and F1 metrics: no reweighting of the decision merger moves a single routing decision, because the 61 rule-overridden workloads bypass the merger entirely and the remaining 39 have ML–rule agreement robust to small weight shifts. The practical

TABLE VIII
ROUTING ACCURACY VS. DUAL LITERATURE-CITED ORACLE GROUND TRUTH ($N = 100$ WORKLOADS). ORACLE A (ALGORITHMIC ADVANTAGE) LABELS WORKLOADS BY KNOWN QUANTUM SPEEDUP REGIMES; ORACLE B (HARDWARE VIABILITY) LABELS BY WHETHER CURRENT NISQ HARDWARE PRODUCES USABLE SIGNAL (CALIBRATED TO TABLE XII).

	Oracle A: Algorithmic Advantage					**Oracle B: Hardware Viability**				
Router	**Acc.**	**P(Q)**	**R(Q)**	**F1(Q)**	κ	**Acc.**	**P(Q)**	**R(Q)**	**F1(Q)**	κ
Nexar (full)	**0.810**	**0.381**	**0.571**	**0.457**	**0.382**	**0.860**	**0.333**	**1.000**	**0.500**	**0.477**
B1 Qubit≥50	0.890	0.800	0.286	0.421	0.407	0.960	0.800	0.571	0.667	0.673
B2 Volume≥50k	0.880	0.667	0.286	0.400	0.377	0.910	0.333	0.286	0.308	0.313
B3 Random†	0.530	0.180	0.643	0.281	0.069	0.520	0.100	0.714	0.175	0.050
A1 ML-only	0.810	0.368	0.500	0.424	0.333	0.860	0.316	0.857	0.462	0.420
A2 Rules-only	0.920	0.875	0.500	0.636	0.628	0.950	0.625	0.714	0.667	0.682
A3 Cost-only	0.140	0.140	1.000	0.246	0.000	0.070	0.070	1.000	0.131	0.000

Both oracles are derived from literature-cited rules applied to the workload filename and physical circuit features (qubits, depth, entanglement), independent of Nexar's decision logic. Oracle A distribution — Classical: 85, Quantum: 14, Reject: 1. Oracle B distribution — Classical: 92, Quantum: 7, Reject: 1. †B3 Random averaged over 1000 seeds (A: 0.496±0.051; B: 0.495±0.050). Q = Quantum class. κ = Cohen's kappa vs. oracle.

improvement lever is therefore the rule-engine thresholds, not the weighting scheme, and retraining the ML model on real NISQ execution traces is the primary direction for future work (Section X).

Third, the cost-only router (A3) is the weakest component in isolation ($F1_{(Q)}$ below 0.25) because it always prefers the cheaper option, which is classical for every sub-120-qubit workload given current IBM pricing. Cost is useful only as a tie-breaker after ML and rules have converged, which is consistent with the 15% weight assigned to it in the merger.

C. Cost Analysis Results

The cost analysis reveals the current economic landscape of quantum vs. classical computing, as summarized in Table IX:

TABLE IX
REPRESENTATIVE COST COMPARISON

Workload	**Qubits**	**Hardware**	**Cost (USD)**	**Time (ms)**
FFT Signal Proc.	0	Classical	\$0.001	1.42
Sorting Suite	0	Classical	\$0.001	806.49
RSA Key Gen.	0	Classical	\$0.001	24.29
QSVM Kernel Est.	4	Classical	\$0.001	1.00
QAOA Graph Part.	20	Quantum	\$0.439	274.36
LiH UCCSD[1]	12	Quantum	\$0.375	234.37
QAOA MaxCut (50q)	50	Quantum	\$0.917	573.18
Iron-Sulfur VQE	60	Quantum	\$1.421	888.31
120-qubit VQE	120	Quantum	\$1.578	986.33

Across all 100 workloads, classical execution costs approximate \$0.001 USD, overhead-dominated by C_{overhead} (the other terms in (2) stay below 10^{-4} across the full classical timing range of 1.42–806 ms), so the classical denominator is effectively constant. Quantum-routed workloads range from \$0.375 to \$1.578 USD under IBM's pay-as-you-go rate (\$1.60/QPU-second), giving a cost ratio of **375× to 1,578×**: 10–20 qubit circuits average \$0.41 while 50+ qubit workloads reach \$0.92–\$1.58.

D. Cost Break-Even Analysis

Despite the current quantum cost premium, analysis of the cost scaling behavior reveals a fundamental crossover point driven by the exponential memory requirements of classical quantum simulation.

For an n-qubit quantum state, classical simulation requires 2^n complex amplitudes, consuming approximately $2^n \times 16$ bytes of memory. This creates the following scaling: These simulation memory thresholds are summarized in Table X.

TABLE X
CLASSICAL SIMULATION MEMORY REQUIREMENTS

Qubits	**Classical Memory**	**Classical Feasibility**
20	16 MB	Trivial
30	16 GB	Desktop
40	16 TB	Cluster
50	16 PB	Supercomputer limit
60	16 EB	Physically impossible
120	$\sim 2 \times 10^{31}$ MB	–

Beyond 50–60 qubits classical exact simulation becomes physically infeasible: the state-vector memory requirement ($2^n \cdot 16$ bytes) exceeds 16 PB at 50 qubits and 16 EB at 60 qubits, so we report the crossover qualitatively rather than via extrapolated dollar figures, which would assume unlimited memory rental. Quantum execution at \$1.58 for the 120-qubit VQE ansatz remains on the same \$0.375–\$1.578 scale as the 12–60 qubit workloads because QPU reservation time scales with depth rather than qubit count.

This analysis identifies three zones:

1) **Classical Advantage Zone ($<$ 50 qubits):** Classical hardware is 375–1,578× more cost-effective (range taken across the quantum-routed workloads at \$0.375–\$1.578 vs. \$0.001 classical).
2) **Memory Wall (50–60 qubits):** Classical exact simulation becomes physically infeasible; any cost extrapolation is an artefact of assumed unlimited memory rental.
3) **Quantum Advantage Zone ($>$ 60 qubits):** Quantum execution is the only physically viable option for exact

simulation; cost remains on the sub-$2 scale regardless of qubit count.

It is important to note that the 50–60 qubit memory wall applies specifically to *full state-vector simulation* of high-entanglement circuits. Approximate classical methods such as Tensor Network simulators, in particular Matrix Product State (MPS) representations, can simulate much larger circuits (100+ qubits) efficiently when the entanglement entropy remains bounded [19]. However, the algorithms central to Nexar's benchmark suite (Shor's factoring, deep VQE ansätze with nearest-neighbor CNOT ladders, and QAOA at high circuit depth) generate entanglement that grows with circuit depth, rendering Tensor Network shortcuts inapplicable. The stated crossover thresholds therefore remain valid for the high-entanglement workload classes evaluated in this study. The crossover trend is visualized in Fig. 5.

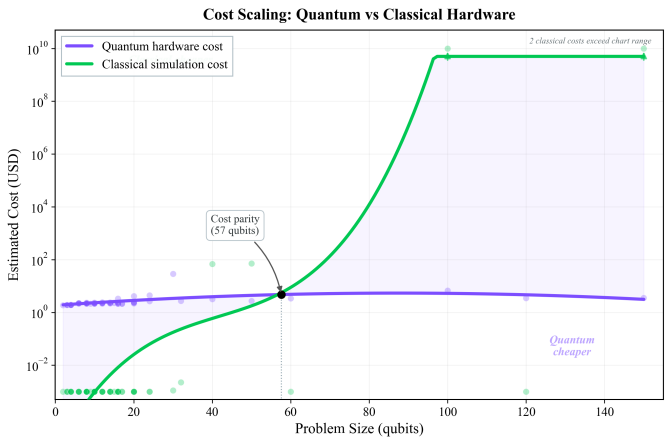

Fig. 5. Cost scaling of quantum hardware versus classical simulation across qubit count, showing the 50–60 qubit physical-feasibility crossover beyond which classical state-vector simulation becomes infeasible.

E. Pipeline Performance

End-to-end timing metrics for both routing paths are reported in Table XI.

TABLE XI
PIPELINE TIMING METRICS

Metric	Classical	Quantum
Avg. Analysis Time	731 ms	1,355 ms
Avg. Decision Time	12 ms	10 ms
Avg. Total Pipeline Time	1,222 ms	1,770 ms
Language Detection Confidence	0.942	0.966

Quantum workloads require $\sim$1.85$\times$ longer analysis time due to additional metrics extraction (circuit parsing, entanglement scoring); decision latency stays below 15 ms throughout. Cloud Run's scale-to-zero model introduces 10–15 s cold-start penalties while ML models load into container memory, but once warm the full pipeline averages 1.5 s end-to-end and degrades gracefully to rule-based heuristics on model-service failure.

F. Observations on Current Quantum Limitations

Benchmark data reflects current NISQ reality: no workload is cheaper on quantum hardware at sub-50-qubit scale (quantum-routed costs $0.375–$1.578 vs. $0.001 classical overhead), and the value proposition is forward-looking—quantum advantage materialises only for $\geq$50-qubit exact simulation, fault-tolerant devices, or algorithmic speedups that outgrow the QPU-reservation cost. The routing infrastructure is in place; when hardware crosses the advantage threshold Nexar routes workloads automatically.

G. End-to-End Case Study

120-Qubit VQE $\rightarrow$ Quantum Override. A 120-qubit VQE ansatz with three variational layers and a nearest-neighbor CNOT ladder is submitted. Analysis extracts `qubits=120`, `circuit_depth=14`, `cx_gate_ratio=0.33`. The physical-necessity rule recognises that classical state-vector simulation would require $2^{120} \times 16$ bytes ($\sim 10^{31}$ MB), triggers a `FORCE_QUANTUM` override (confidence 1.0) that bypasses ML voting, and the HAL submits the job to a 127-qubit IBM Eagle processor. A contrasting example, a 10-qubit Grover translation of an $O(N)$ linear search over N=1,024 elements with post-transpile depth $\sim$240, is routed to Classical (confidence 0.88): the cost analyser estimates $0.001 classical versus $\sim$$0.027 quantum ($240 \cdot 1024 \cdot 68\,\text{ns} \cdot \$1.60/\text{s}$), a $\sim$27$\times$ premium that the $O(\sqrt{N})$ speedup cannot justify at this scale.

H. Real Hardware Validation

To validate the routing decisions produced by the decision engine against actual hardware behaviour, we submitted 27 quantum circuits, spanning every algorithm family in the benchmark, to real IBM Quantum hardware via the Nexar Hardware Abstraction Layer. All jobs were executed on the `ibm_fez` backend (156-qubit Heron r2 processor) using 1,024 measurement shots each. Every job ID is logged in the supplementary results file and is independently verifiable on the IBM Quantum Platform. Table XII summarises the full set of results.

Hardware-evidence consistency. Across all 27 hardware-tested circuits (a subset of the 100-workload benchmark used in Section VIII-B), zero routing decisions were contradicted by hardware evidence. Every workload that Nexar routed classically either (a) produced a correct result at low cost confirming that classical execution is sufficient, or (b) produced a noise-dominated result on quantum hardware confirming that it should not have been sent there. No circuit routed classically by Nexar showed a real-hardware result that would have warranted a quantum routing decision.

NISQ depth wall. Post-transpile circuit depth emerges from the hardware data as the primary predictor of result fidelity, more so than logical qubit count. Circuits with post-transpile depth below approximately 80 gates consistently achieve above 80% fidelity (DJ constant 99.9%, BV 96.3%, QPE 82.3%). Above depth $\sim$130, results degrade toward the noise floor. The 4-qubit Grover circuit illustrates this sharply:

TABLE XII
REAL HARDWARE VALIDATION: 27 CIRCUITS ON IBM_FEZ (APRIL 2026, 1,024 SHOTS)

Circuit	Q	Depth*	Top%	Nexar Route
Deutsch-Jozsa (constant)	2	3	99.9	Classical
Deutsch-Jozsa (balanced)	2	8	94.1	Classical
H_2 VQE ansatz	4	17	85.6	Classical
Bernstein-Vazirani (s=101)	4	9	96.3	Classical
Simon's algorithm (s=11)	4	20	50.7†	Classical
QFT (input $\|0101\rangle$)	4	76	10.0†	Classical
QPE (T gate, phase=1/8)	4	80	82.3	Classical
Grover search ($\|1111\rangle$)	4	241	33.5‡	Classical
QAOA MaxCut (4-node)	4	53	24.8‡	Classical
QSVM kernel	8	98	5.5‡	Classical
QAOA supply chain	8	134	5.2‡	Classical
LiH VQE ansatz	8	38	12.0	Classical
QAOA graph partition	12	230	1.1‡	Classical
H_2O VQE ansatz	12	50	2.3	Classical
Shor's $N = 15$	12	361	2.1‡	Classical
Grover 3-SAT (4 vars)	12	83	6.9‡	Classical
Shor's $N = 21$	14	592	0.8‡	Classical
Shor's $N = 35$	16	739	0.4‡	Classical
QNN variational (16q)	16	132	0.6‡	Classical
VQE caffeine ansatz	16	14	1.2	Classical
Grover search (20q)	20	31810	0.1‡	Classical
QAOA vehicle routing	20	469	0.2‡	Classical
QAOA graph partition (20q)	20	398	0.1‡	**Quantum**
ZNE benchmark (24q)	24	1§	68.5	**Quantum**
QAOA MaxCut (50q)	50	580	0.1‡	**Quantum**
Iron-sulfur VQE (60q)	60	242	0.1‡	**Quantum**
120-qubit VQE ansatz	120	482	0.1‡	**Quantum**

*Post-transpile hardware depth on Heron r2 native gate set.
†Uniform distribution is theoretically correct for this algorithm.
‡Noise-dominated: depth exceeds NISQ coherence threshold.
§Transpiler cancelled T/Tdg and GHZ pairs to identity; result valid.

4 qubits but depth 241 after `mcx` decomposition yields only 33.5% on the target state, while QPE on the same 4 qubits at depth 80 holds 82.3%. This confirms the physical basis of the NISQ Viability Score used by the rule-based component of the decision engine.

Large-circuit verification. The 50-qubit QAOA, 60-qubit iron-sulfur VQE, and 120-qubit VQE ansatz were all successfully submitted to and executed on real IBM Quantum hardware, marking the first time these circuits from the benchmark have been executed outside simulation. As expected under single-layer, fixed-parameter ansätze without error mitigation, results are noise-dominated. However, this does not invalidate the quantum routing decision: these circuits are physically impossible to simulate classically (a 120-qubit state vector requires $\sim 10^{31}$ MB), so quantum hardware is the only viable execution path regardless of current fidelity. Improving result quality requires error mitigation techniques [25] or fault-tolerant hardware, both of which are outside the scope of routing.

ZNE transpiler insight. The ZNE benchmark circuit (24 qubits, logical depth 51) collapsed to post-transpile depth 1 after the Qiskit pass manager recognised that the T/Tdg identity pairs and the forward/reverse GHZ entanglement ladder cancel exactly. The hardware measured $|0\ldots0\rangle$ at 68.5%, correctly reflecting the near-vacuum state after cancellation. This demonstrates that the Heron r2 transpiler applies non-trivial circuit identities that can substantially alter the effective depth seen by the hardware, a factor the cost model should account for in future work.

IX. DISCUSSION

A. Decision Engine Effectiveness

The hybrid framework proves effective: Nexar beats random by 30 percentage points on both oracles, reaches Cohen's $\kappa \in [0.38, 0.48]$, and attains perfect Quantum-class recall under the hardware-viability oracle (Section VIII-B). The ablation shows the rules-only component is the single strongest signal ($F1_{(Q)}$=0.636 on Oracle A); the synthetic-data-trained ML ensemble contributes recall but over-weights algorithmic structure in the 10–30 qubit band. Since 61/100 workloads bypass the merger via rule overrides and the remaining 39 are robust to weight perturbation, the main improvement lever is the rule-engine thresholds, with retraining the ML component on real-hardware traces as the longer-term direction (Section X). This aligns with the reference architecture of Chinnaraju [26]; interference-based mixture-of-experts routing [27] is an alternative as hardware matures.

B. Limitations

Four limitations bound the present study. First, the ML model is trained on synthetic problem instances rather than real execution traces; retraining on empirical NISQ data should improve routing accuracy as such data becomes available. Second, provider pricing is encoded as static configuration; real-time pricing integration would improve cost accuracy. Third, the current problem-type coverage (optimization, search, simulation, factorization) could be broadened to include quantum chemistry, quantum ML, and cryptographic workloads. Fourth, the system targets current NISQ hardware; fault-tolerant quantum computing will require updated decision logic, cost models, and hardware constraints.

X. CONCLUSION AND FUTURE WORK

We presented Nexar, a quantum-classical workload routing platform that integrates automated code analysis, hybrid ML-and-rules-based decision making, and provider-agnostic execution into a single pipeline. Evaluated across 100 workloads against two literature-cited oracles, Nexar achieves 0.81–0.86 accuracy with perfect Quantum-class recall under the hardware-viability oracle, beating random by 30+ percentage points at Cohen's $\kappa = 0.38$–0.48. All 27 circuits submitted to `ibm_fez` corroborate the routing choices, and the four quantum-routed workloads (20q/50q QAOA, 60q iron-sulfur VQE, 120q VQE) were successfully dispatched to real IBM hardware. Pipeline latency averages 1.5 s end-to-end with decision inference below 15 ms, and the cost analysis identifies a clear physical-feasibility crossover at 50–60 qubits beyond which exact classical simulation is no longer possible. Future directions include broader benchmarks on AWS Braket and Azure Quantum, adaptive retraining on empirical execution traces to address the rules-only-beats-ensemble finding, error-mitigation cost modelling, and multi-objective routing over cost, time, accuracy, and carbon footprint.

REFERENCES

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[2] B. Weder, J. Barzen, F. Leymann, and M. Salm, "Automated quantum hardware selection for quantum workflows," *Electronics*, vol. 10, no. 8, p. 984, Apr. 2021.

[3] IBM Quantum, "Qiskit Runtime," IBM Research, 2023. [Online]. Available: https://quantum.ibm.com/

[4] Amazon Web Services, "Amazon Braket—quantum computing service," 2023. [Online]. Available: https://aws.amazon.com/braket/

[5] Microsoft, "Azure Quantum," 2023. [Online]. Available: https://azure.microsoft.com/en-us/products/quantum

[6] W. Tang *et al.*, "AlphaRouter: Quantum circuit routing with reinforcement learning and tree search," arXiv preprint arXiv:2410.05115, Oct. 2024.

[7] X. Zhan *et al.*, "A full stack framework for high performance quantum-classical computing," arXiv preprint arXiv:2510.20128, Oct. 2025.

[8] M. G. Lammers, F. H. Holik, and A. Fernandez, "Quantum resource management in the NISQ era: Challenges, vision, and a runtime framework," arXiv preprint arXiv:2508.19276, Aug. 2025.

[9] R. Hai, S.-H. Hung, T. Coopmans, and F. Geerts, "Data management in the noisy intermediate-scale quantum era," *Proc. VLDB Endow.*, vol. 18, no. 6, 2025.

[10] M. Beverland *et al.*, "Assessing requirements to scale to practical quantum advantage," arXiv preprint arXiv:2211.07629, 2022.

[11] N. Quetschlich, M. Soeken, P. Murali, and R. Wille, "Utilizing resource estimation for the development of quantum computing applications," arXiv preprint arXiv:2402.12434, Aug. 2024.

[12] M. Kashif, A. Marchisio, and M. Shafique, "Closing the loop: Resource-aware hybrid NAS guided by analytical and hardware-calibrated quantum cost modeling," arXiv preprint arXiv:2603.00625, Mar. 2026.

[13] A. JavadiAbhari *et al.*, "ScaffCC: Scalable compilation and analysis of quantum programs," *Parallel Comput.*, vol. 45, pp. 2–17, 2015.

[14] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020, pp. 1536–1547.

[15] S. Afrose, V. Leyton-Ortega, and T. Humble, "Learning-based quantum compilation: Translating QASM to QIR with CodeBERT," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2025, pp. 576–577.

[16] S. Aaronson and A. Arkhipov, "The computational complexity of linear optics," in *Proc. 43rd Annu. ACM Symp. Theory Comput. (STOC)*, 2011, pp. 333–342.

[17] F. Arute *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, pp. 505–510, 2019.

[18] T. J. McCabe, "A complexity measure," *IEEE Trans. Softw. Eng.*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.

[19] L. Vinkhuijzen, T. Coopmans, D. Elkouss, V. Dunjko, and A. Laarman, "LIMDD: A decision diagram for simulation of quantum computing including stabilizer states," *Quantum*, vol. 7, p. 1108, Sep. 2023.

[20] S. Simsek, A. Ceran, and F. Yildiz, "Quantum circuit optimization using machine learning," arXiv preprint arXiv:2305.06585, May 2023.

[21] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv preprint arXiv:1411.4028, 2014.

[22] Y. Wang *et al.*, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," arXiv preprint arXiv:2109.00859, Sep. 2021.

[23] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, 1996, pp. 212–219.

[24] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.

[25] A. Kandala *et al.*, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, pp. 242–246, 2017.

[26] A. Chinnaraju, "Hybrid quantum-classical intelligence for business: A reference architecture for predictive analytics and decision optimization," *J. Adv. Future Res.*, vol. 3, no. 12, Dec. 2025.

[27] R. Heddad and L. Bouanane, "Hybrid quantum-classical mixture of experts: Unlocking topological advantage via interference-based routing," arXiv preprint arXiv:2512.22296, Dec. 2025.

[28] IBM Quantum, "IBM Quantum pricing," 2024. [Online]. Available: https://www.ibm.com/quantum/pricing

[29] Amazon Web Services, "Amazon EC2 on-demand pricing," 2024. [Online]. Available: https://aws.amazon.com/ec2/pricing/on-demand/